Calculadora de Edad Exacta: Revisión Integral en Python y C++ con Validación Calendárica y Filtros de Consistencia

Aramis Demaria

E.E.S.T. N.º 1 “Eduardo Ader” – Vicente López

Laboratorio de Programación (LPR) – 5.º Año 3.º A-B

Prof. Mansilla Muñoz York Elías

2026

Resumen

Este informe documenta el diseño, la implementación y la validación de una calculadora de edad exacta que determina la edad de una persona al 8 de julio de 2026 a partir de su fecha de nacimiento. El sistema valida la existencia de la fecha en el calendario gregoriano, incluido el tratamiento de febrero en años bisiestos, y ajusta la edad cuando la persona aún no ha cumplido años en el año de referencia. La solución se desarrolló en paralelo en Python y en C++ para comparar legibilidad, tipado estático y dinámico, manejo de errores y flujo de compilación-ejecución. El algoritmo se descompone en las funciones *esBisiesto* y *esFechaValida*, siguiendo el principio de programación defensiva con un filtro de consistencia que rechaza días, meses y años fuera de rango. Se diseñaron pruebas de borde (e.g., 29/02/2000, 29/02/1900, 31/04/2005, 08/07/2010 y 09/07/2010) que confirman la coincidencia de ambas implementaciones y la correcta aplicación de la regla gregoriana del bisiesto. Los resultados muestran que la modularización reduce errores y facilita el mantenimiento, y que la validación temprana en la entrada es más eficiente que la corrección posterior.

Palabras clave: edad exacta, año bisiesto, filtro de consistencia, C++, Python, validación, scaffolding

Abstract

This report documents the design, implementation, and validation of an exact age calculator that determines a person’s age as of July 8, 2026. The system validates the existence of the birth date in the Gregorian calendar, including February in leap years, and adjusts the age when the birthday has not yet occurred in the reference year. The solution was implemented in both Python and C++ to compare readability, static versus dynamic typing, error handling, and the compile-and-run workflow. The algorithm is decomposed into the functions *esBisiesto* and *esFechaValida*, applying defensive programming through a consistency filter that rejects out-of-range days, months, and years. Edge-case tests (e.g.,

02/29/2000, 02/29/1900, 04/31/2005, 07/08/2010, and 07/09/2010) confirm agreement between both implementations and correct application of the Gregorian leap-year rule.

Keywords: exact age, leap year, consistency filter, C++, Python, validation, scaffolding

Introducción

La primera actividad del ciclo lectivo 2026 propone un repaso de la lógica y los algoritmos trabajados en cuarto año, contrastando un lenguaje interpretado de alto nivel (Python) con un lenguaje compilado estructurado (C++). El problema de calcular la edad exacta al 8 de julio de 2026 es aparentemente sencillo, pero exige rigor en la validación calendárica. El objetivo general del trabajo es implementar filtros de consistencia que impidan el ingreso de fechas inexistentes (por ejemplo, 31 de abril o 30 de febrero) y aplicar correctamente la regla gregoriana del año bisiesto. El objetivo específico es comparar la expresividad, el control de tipos y el tratamiento de errores entre ambas implementaciones.

El proyecto se desarrolla bajo control de versiones con Git, respetando la estructura */calculadoraedad* solicitada en la guía: *proyecto_cpp/*, *proyecto_python/* y *docs/*. Cuando las computadoras del laboratorio bloquean permisos de administrador para compilar localmente con MSYS2/MinGW, se utiliza la alternativa OnlineGDB, conservando en todos los casos el repositorio en GitHub y las capturas de ejecución. El presente informe se organiza según las normas de la séptima edición del Publication Manual of the American Psychological Association (American Psychological Association [APA], 2020).

Revisión de la Literatura

La validación de fechas es un ejemplo clásico de programación defensiva: nunca se debe confiar en la entrada del usuario y el rechazo temprano (*fail-fast*) es menos costoso que la corrección de datos corruptos en etapas posteriores (Stroustrup, 2013). La regla gregoriana del bisiesto establece que un año es bisiesto si es divisible por 4 y no por 100, excepto cuando es divisible por 400 (Van Rossum & Drake, 2009). Febrero tiene 29 días en

años bisiestos y 28 en caso contrario, mientras que abril, junio, septiembre y noviembre tienen 30 días y el resto 31. En Python, la tipificación dinámica y el bloque *try/except* simplifican la validación de tipo; en C++, la tipificación estática y el código de salida del programa (*return 1*) ofrecen control explícito sobre el estado de terminación (Microsoft, 2024).

El concepto de *scaffolding* o esqueleto, entendido como la plantilla con estructura base provista por el docente, permite al estudiante concentrarse en la lógica faltante sin dedicar esfuerzo al código repetitivo (*boilerplate*). La edad exacta se define como *edad = añoActual − añoNacimiento*, con la corrección *si (mesNacimiento > mesActual) o (mesNacimiento = mesActual y díaNacimiento > díaActual) entonces edad = edad − 1*, para evitar sobreestimar la edad de quien aún no ha cumplido años en el año de referencia.

Método

Diseño

Se adoptó un enfoque estructurado descendente (*top-down*), descomponiendo el problema en dos funciones puras —*esBisiesto* y *esFechaValida*— y un programa principal que coordina la entrada, la validación y el cálculo. El esqueleto provisto ya contenía los filtros base; el estudiante completó el condicional de ajuste de edad. La fecha de referencia se fijó en 8 de julio de 2026 para todos los ensayos, conforme a la consigna.

Materiales e Instrumentos

Los instrumentos fueron: (a) Visual Studio Code como editor, (b) compilador GCC (g++) 13.2.0 mediante MSYS2/MinGW en Windows, con verificación mediante *g++ --version* y variable de entorno PATH, (c) intérprete Python 3.12, y (d) repositorio Git con archivo *.gitignore* para excluir ejecutables y carpeta *.vscode*. Como alternativa para equipos del laboratorio sin permisos de administrador, se contempló OnlineGDB (https://www.onlinegdb.com).

Procedimiento

El procedimiento siguió cinco pasos: (1) lectura de día, mes y año de nacimiento; (2) validación de rango de año (1900–2026), mes (1–12) y día según mes, considerando bisiesto; (3) si la fecha es inválida, emisión de mensaje de error y terminación con código 1 en C++ o mensaje de error en Python; (4) cálculo de edad bruta y ajuste por mes y día; y (5) presentación de la fecha actual, la fecha ingresada, la indicación de bisiesto cuando corresponde y la edad calculada. La estructura de carpetas se creó como:

```
/calculadoraedad/
  ■■■ .gitignore
  ■■■ README.md
  ■■■ LICENSE
  ■■■ docs/InformeProyectoCalculadoraEdad.pdf
  ■■■ proyecto_cpp/calculadoraEdad.cpp
  ■■■ proyecto_python/calculadoraEdad.py
```

La compilación y ejecución se realizaron con los comandos *g++ proyecto_cpp/calculadoraEdad.cpp -o proyecto_cpp/calculadoraEdad.exe* seguido de *.\proyecto_cpp\calculadoraEdad.exe* para C++, y *python proyecto_python/calculadoraEdad.py* para Python.

Resultados

Ambas implementaciones compilaron y ejecutaron correctamente y coincidieron en todos los casos de prueba. El filtro rechazó fechas como 32 de enero y 31 de junio, aceptó 29/02/2004 y rechazó 29/02/2003. El ajuste de edad funcionó según lo previsto: una persona nacida el 10/07/2008 al 08/07/2026 obtuvo 17 años en lugar de 18, al no haber cumplido años aún. La Tabla 1 sintetiza la comparativa sintáctica entre lenguajes y la Tabla 2 los casos de borde.

Tabla 1

Comparativa Sintáctica entre Python y C++ para el Mismo Algoritmo

Concepto	Python	C++
Tipado	Dinámico (tipo inferido)	Estático (int explícito)
Ejecución	Interpretado (python .py)	Compilado (g++ → .exe)

Concepto	Python	C++
Manejo de error de tipo	try / except ValueError	Validación booleana y return 1
Entrada	input() + int()	cin >>

Nota. Elaboración propia con base en la documentación oficial de ambos lenguajes.

Tabla 2

Casos de Borde para Validación Calendárica y Cálculo de Edad

Fecha de nacimiento	Esperado	Resultado
29/02/2000	Válida (divisible por 400)	Válida – 26 años
29/02/1900	Inválida (siglo no divisible por 400)	Rechazada
31/04/2005	Inválida (abril 30 días)	Rechazada
08/07/2010	16 años (cumple hoy)	16 años
09/07/2010	15 años (aún no cumple)	15 años

Nota. Fecha de referencia: 08/07/2026. La edad se ajusta restando uno cuando el cumpleaños aún no ha ocurrido.

El desempeño computacional es despreciable para este dominio; la relevancia es didáctica. Python redujo aproximadamente 30% las líneas por su sintaxis compacta, mientras que C++ aportó control explícito de tipos y códigos de salida del sistema operativo. La evidencia de ejecución con el nombre del estudiante en la cabecera satisface la regla antiplagio de la consigna.

Discusión

Los hallazgos confirman que validar en la entrada (*fail-fast*) es menos costoso que corregir datos corruptos en etapas posteriores. La fórmula gregoriana del bisiesto es el ejemplo prototípico de condición compuesta con operadores módulo, conjunción y disyunción. La corrección condicional de la edad —el bloque que el estudiante debía completar— es el núcleo conceptual de la actividad y se implementó correctamente en ambos lenguajes.

Una limitación del presente trabajo es el rango arbitrario 1900–2026, fijado con fines didácticos; en un sistema productivo se emplearía la fecha del sistema (*time.h* en C++

o *datetime* en Python). En C++, la validación de entrada no numérica más allá del fallo de *cin* requeriría verificación adicional del estado del flujo, mientras que en Python se captura la excepción *ValueError*. La técnica de validación es reutilizable para cualquier formulario con fechas, como registros de sensores, turnos o vencimientos del Proyecto Integrador Anual (PIA) para la Expotécnica.

En el plano formativo, el scaffolding permitió concentrar el esfuerzo cognitivo en la lógica de ajuste de edad, reduciendo errores de sintaxis iniciales. La comparación inter-lenguaje hizo visible que, aunque la lógica es la misma, la gestión de errores y el ciclo de compilación difieren sustancialmente.

Conclusión

Se logró una calculadora de edad exacta, validada y portable en dos lenguajes, respetando la estructura de carpetas y el versionado con Git. El ajuste condicional de edad se implementó conforme a la especificación y superó las pruebas de borde. Como trabajo futuro, se propone extraer la lógica a una biblioteca compartida y agregar pruebas unitarias automatizadas. La experiencia sienta bases para el PIA y para actividades posteriores de modularización, punteros y programación orientada a objetos.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

Microsoft. (2024). *Documentación de C++: Tipos y operadores*. https://learn.microsoft.com/es-es/cpp/cpp/

Python Software Foundation. (2024). *Python docs: Input and output*. https://docs.python.org/3/tutorial/inputoutput.html

Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.

Van Rossum, G., & Drake, F. L. (2009). *The Python language reference*. Python Software Foundation.

Apéndice A

Código Fuente Completo

Listado A1. C++ (proyecto_cpp/calculadoraEdad.cpp)

```
#include <iostream>
using namespace std;

bool esBisiesto(int anio) {
return (anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0);
}
bool esFechaValida(int d, int m, int a) {
if (a < 1900 || a > 2026) return false;
if (m < 1 || m > 12) return false;
if (d < 1 || d > 31) return false;
if ((m == 4 || m == 6 || m == 9 || m == 11) && d > 30) return false;
if (m == 2) {
if (esBisiesto(a)) { if (d > 29) return false; }
else { if (d > 28) return false; }
}
return true;
}
int main() {
int diaN, mesN, anioN;
int diaA = 8, mesA = 7, anioA = 2026;
cout << "CALCULADORA DE EDAD DE: Aramis Demaria" << endl;
cin >> diaN >> mesN >> anioN;
if (!esFechaValida(diaN, mesN, anioN)) {
cout << "[ERROR] La fecha no existe." << endl; return 1;
}
int edad = anioA - anioN;
if (mesN > mesA || (mesN == mesA && diaN > diaA)) edad--;
if (edad < 0) edad = 0;
cout << "Edad calculada: " << edad << " anios." << endl;
return 0;
}
```

Listado A2. Python (proyecto_python/calculadoraEdad.py)

```
def es_bisiesto(anio):
return (anio % 4 == 0 and anio % 100 != 0) or (anio % 400 == 0)

def es_fecha_valida(d, m, a):
if a < 1900 or a > 2026: return False
if m < 1 or m > 12: return False
if d < 1 or d > 31: return False
if m in [4, 6, 9, 11] and d > 30: return False
if m == 2:
if es_bisiesto(a):
if d > 29: return False
else:
if d > 28: return False
return True

print("CALCULADORA EN PYTHON DE: Aramis Demaria")
try:
diaN = int(input("Dia: ")); mesN = int(input("Mes: "))
anioN = int(input("Anio: "))
diaA, mesA, anioA = 8, 7, 2026
```

```
if not es_fecha_valida(diaN, mesN, anioN):
print("[ERROR] Fecha invalida.")
else:
edad = anioA - anioN
if mesN > mesA or (mesN == mesA and diaN > diaA):
edad -= 1
print(f"Edad calculada: {edad} anios.")
except ValueError:
print("[ERROR] Deben ingresar enteros.")
```

Apéndice B

Evidencia de Ejecución

La ejecución se verifica con *g++ proyecto_cpp/calculadoraEdad.cpp -o proyecto_cpp/calculadoraEdad.exe* y *python proyecto_python/calculadoraEdad.py*. En equipos sin permisos de administrador se emplea OnlineGDB, adjuntando capturas de la terminal y del repositorio GitHub con la estructura completa.